Vendora – AI integrated eCommerce Platform

Final Submission (Phase 2)

Software Requirement Engineering x Software Project Management

Group 4

Submitted to Dr. Muddassira Arshad

Project: Vendora – AI integrated eCommerce Platform

Phase: Phase 2 --- Final Submission

Date: 09-05-2026

GitHub Link：https://github.com/Als4742/ai-powered-web-assistant.git

Application Link: https://vendora2.lovable.app/

Figma Design Link

https://www.figma.com/design/U7qfoCBHkuR4btvBGHJREd/Untitled?node-id=0-1&t=ForVpvKSYeNUUdAl-1

Group Members & Roles:

#	Roll No.	Student Name	Gender	Class	Role
1	BSEF23M502	Iraj Fatima	Female	SE-F23	Project Manager & Governance Lead
2	BSEF23M516	Ahmad Saleem	Male	SE-F23	Full Stack Developer & Project Architect
3	BSEF23M520	Misal Fatima	Female	SE-F23	Control Lead & DevOps/Risk Lead
4	BSEF24M537	M. Hasnain Asif	Male	SE-F24	Requirements Lead & Backend Lead
5	BSEF24M536	Muhammad Saad	Male	SE-F24	Technical Lead & AI Specialist
6	BSEF24M535	Safa Saeed	Female	SE-F24	Validation Lead & QA Lead

FEATURE DESCRIPTION

1. Introduction

1.1 Purpose

This document describes the functional features of **Vendora**, an AI-integrated eCommerce platform. It provides a clear understanding of how each feature works and how users—both sellers and buyers—interact with the system to facilitate a streamlined online selling and shopping experience.

1.2 Scope

Vendora is a web-based Minimum Viable Product (MVP) designed to reduce operational friction for small-scale sellers. The system supports two primary roles: **Sellers**, who manage product listings and orders, and **Buyers**, who browse and purchase items. The application leverages AI to automate content creation and enhance customer engagement.

1.3 Definitions

- **Vendora:** The AI-integrated eCommerce Store system.
- **Seller:** The administrative user responsible for product management and order fulfillment.
- **Buyer:** The end-user who interacts with the storefront to purchase goods.
- **MVP:** Minimum Viable Product, focusing on core essential features within a 30-day delivery limit.
- **CRUD:** Create, Read, Update, and Delete operations for data management.

2. System Overview

2.1 System Users

- **Sellers:** Utilize a dedicated dashboard to list products, generate AI-driven descriptions, and track incoming orders.
- **Buyers (Attendees):** Use the public-facing storefront to browse the catalog, manage a shopping cart, and complete the checkout process.

2.2 System Environment

- Web-based application
- Runs in modern browser

3. Feature Descriptions

3.1 User Authentication & Account Management

3.1.1 Description

This feature manages seller identity and secures access to the private management dashboard.

3.1.2 Functional Requirements

- Sellers can log in using email and password.
- The system issues a secure session token upon successful authentication.
- Sellers can securely log out to terminate the active session.
- Access to seller routes and API endpoints is restricted to authenticated users only.

3.1.3 System Behavior

- Incorrect credentials → Login denied with an error message.
- Non-authenticated access attempt → System redirects to login or returns an "Unauthorized" response.
- Successful logout → Session token is invalidated immediately.

3.2 Product Management (Seller Side)

3.2.1 Description

Allows sellers to create and maintain their product catalog through a unified interface.

3.2.2 Functional Requirements

- Create, Read, Update, and Delete (CRUD) product listings.
- Upload up to 3 product images per item (maximum 2MB per image).
- Toggle stock status (e.g., In Stock vs. Out of Stock).
- Store metadata including title, price, and category.

3.2.3 System Behavior

- Deleted products are immediately removed from the buyer storefront.
- Updates to pricing or details reflect in real-time on the public catalog.
- System prevents saving products without mandatory fields (e.g., Title, Price)

3.3 Product Browsing & Discovery (Buyer Side)

3.3.1 Description

Enables buyers to explore products and find specific items via the storefront.

3.3.2 Functional Requirements

- View a responsive grid-layout list of all active products.
- Access individual product detail pages with full descriptions and images.
- Basic keyword search functionality to filter products by title or keywords.

3.3.3 System Behavior

- Only products marked "In Stock" display an "Add to Cart" option.
- Search results update dynamically based on keyword matching.
- Grid layout adjusts responsively for mobile and desktop views.

3.4 Shopping Cart System

3.4.1 Description

Allows buyers to manage a selection of items before proceeding to checkout.

3.4.2 Functional Requirements

- Add products to a client-side shopping cart.
- Update item quantities or remove items from the cart.
- Calculate and display subtotal and total costs in real-time.

3.4.3 System Behavior

- Cart remains persistent throughout the browsing session.
- Adding the same item multiple times increments the quantity.
- Total cost updates automatically whenever quantities change.

3.5 Order Capture & Checkout System

3.5.1 Description

Handles the transition from shopping to a confirmed order for the seller to fulfill.

3.5.2 Functional Requirements

- Capture buyer details (Name, Email, Delivery Address).
- Generate an order summary for the buyer upon completion.
- Store order data in the database with a "Pending" status.

3.5.3 System Behavior

- Checkout is blocked if the cart is empty.
- Successful submission clears the buyer's shopping cart.
- New orders immediately appear on the Seller's order management table.

3.6 Seller Dashboard (Order Management)

3.6.1 Description

Provides sellers with a central interface to monitor business activity and fulfillment.

3.6.2 Functional Requirements

- View a chronological list of all customer orders.
- Update order fulfillment status (e.g., Pending, Shipped, Delivered).
- Monitor core inventory levels through the product table.

3.6.3 System Behavior

- Dashboard is accessible only by authenticated Sellers.
- Status updates are saved to the central database (Supabase) for record-keeping.

3.7 AI-Assisted Features

3.7.1 Description

Utilizes Large Language Models (LLMs) to automate content creation and product cross-selling.

3.7.2 Functional Requirements

- **AI Description Assistant:** Generate professional product descriptions from a few seller-provided keywords.
- **Manual Override:** Sellers can edit or rewrite any AI-generated text.
- **Intelligent Suggestion Engine:** Automatically display two related items on product pages based on category/metadata.

3.7.3 System Behavior

- AI generation is optional; sellers can choose to write descriptions manually.
- Suggestions are generated dynamically to keep buyers engaged on the site.

3.8 Navigation & User Interface

3.8.1 Description

Defines the visual and interactive framework of the Vendora platform.

3.8.2 Functional Requirements

- Clean, minimalist "Pinterest-style" aesthetic.
- Navigation menu for Buyers (Home, Search, Cart).
- Navigation menu for Sellers (Dashboard, Products, Orders, Logout).
- Single Page Application (SPA) behavior using React.js.

3.8.3 System Behavior

- Fast navigation without full page reloads.
- Loading indicators display during AI generation or image uploads to provide feedback.

4. Feature Interaction Flow

- **Seller Authentication:** Seller logs into the management dashboard using secure credentials to access administrative tools.
- **Product Creation & AI Assistance:** Seller initiates a new listing; the AI Description Assistant generates professional content based on provided keywords, which the seller then reviews and approves.
- **Buyer Browsing:** A buyer visits the storefront, searches for products via the keyword search, and views individual product detail pages.
- **Cart & Recommendation Loop:** Buyer adds an item to the cart; the Suggestion Engine dynamically displays two related products to encourage further exploration.
- **Checkout & Order Capture:** Buyer completes the checkout form with delivery details; the system transitionally saves the order data and clears the buyer's cart.
- **Seller Fulfillment:** Seller views the new order on their dashboard and updates the status:
  - **Shipped/Delivered** → Order cycle completed.
  - **Cancelled** → Inventory remains adjusted and order is archived.

5. Feature Dependencies

- **Dashboard Access depends on Authentication:** The product management and order tracking interfaces are strictly inaccessible without a valid session token.
- **AI Generation depends on Keyword Input:** The Description Assistant requires a minimum set of seller-provided keywords to produce relevant marketing copy.
- **Checkout depends on Cart Validation:** The order capture process cannot be initiated unless the shopping cart contains at least one "In Stock" item.
- **Suggestion Engine depends on Product Metadata:** Related product recommendations rely on existing product categories and tags stored in the database to ensure relevance.
- **Order Fulfillment depends on Data Persistence:** The seller's ability to manage orders is dependent on the successful capture and storage of buyer information in the Supabase database during the checkout flow.

6. Constraints

- **Fixed Delivery Timeline (30 Days):** The project must be completed within a strict 5-week window, requiring disciplined focus on the Minimum Viable Product (MVP) features and preventing scope creep.
- **Resource & Tooling Limits:** Development is restricted to free-tier services and open-source tools (e.g., Supabase free tier, Vercel/Netlify for deployment) to keep operational costs at zero.
- **Manual Fulfillment Workflow:** The system does not include automated payment gateways or automated shipping; order status updates and fulfillment tracking are managed manually by the seller.
- **Lightweight AI Implementation:** AI capabilities are limited to API-based text generation (LLMs) for product descriptions and a rule-based logic for the suggestion engine, rather than complex, custom-trained models.
- **Web-Only Platform:** The system is constrained to a web-based environment (Next.js), with no support for native mobile applications (iOS/Android).

7. Assumptions

- **Stable Internet Connectivity:** It is assumed that both sellers and buyers have consistent access to the internet to interact with the web-based storefront and dashboard.
- **Active Seller Management:** The business logic assumes the seller will regularly check the dashboard to process pending orders and update stock levels.
- **Data Accuracy:** The system assumes that sellers provide valid product metadata and that buyers provide accurate delivery details during the checkout process.

- **Standard Authentication Sufficiency:** It is assumed that standard email/password authentication provided by Supabase is sufficient for the security needs of this MVP without requiring custom encryption logic.
- **High AI Value Delivery:** The project assumes that AI-generated descriptions and simple recommendations will meaningfully improve the user experience for small-scale sellers.

8. Conclusion

This document defines all major features of the **Vendora** AI-integrated eCommerce platform. It ensures clarity in system functionality and supports development, testing, and evaluation processes to ensure the delivery of a high-quality Minimum Viable Product (MVP).

Tech Stack and Reason of Selection

Category	Technology	Reason of Selection
Frontend Technology	**React 19 + TanStack Start**	Uses a component-based architecture for reusable UI elements (product cards, seller dashboard). TanStack Start provides a full-stack framework with Server-Side Rendering (SSR) to ensure fast initial page loads and high-end performance.
	Tailwind CSS v4	Provides ultra-fast styling through utility classes. Configured via CSS variables in **oklch** color space, enabling a modern, vibrant, and "Pinterest-like" aesthetic with minimal code.
	shadcn/ui + Radix UI	Offers professional, accessible, and high-end UI primitives (Dialogs, Selects, Dropdowns). This ensures a "luxury editorial" feel while significantly speeding up development through pre-built accessible components.
	Lucide React	A clean, consistent icon library that aligns with the minimalist and modern visual theme of the Vendora platform.
Backend Technology	**Cloudflare Workers**	Operates on an edge runtime, ensuring the application is globally fast. It provides a lightweight, scalable server environment that integrates seamlessly with the Vite-based build process.
	TanStack Query	Manages asynchronous data fetching and state synchronization. It ensures the UI stays updated without manual page refreshes when a seller adds a product or a buyer updates their cart.

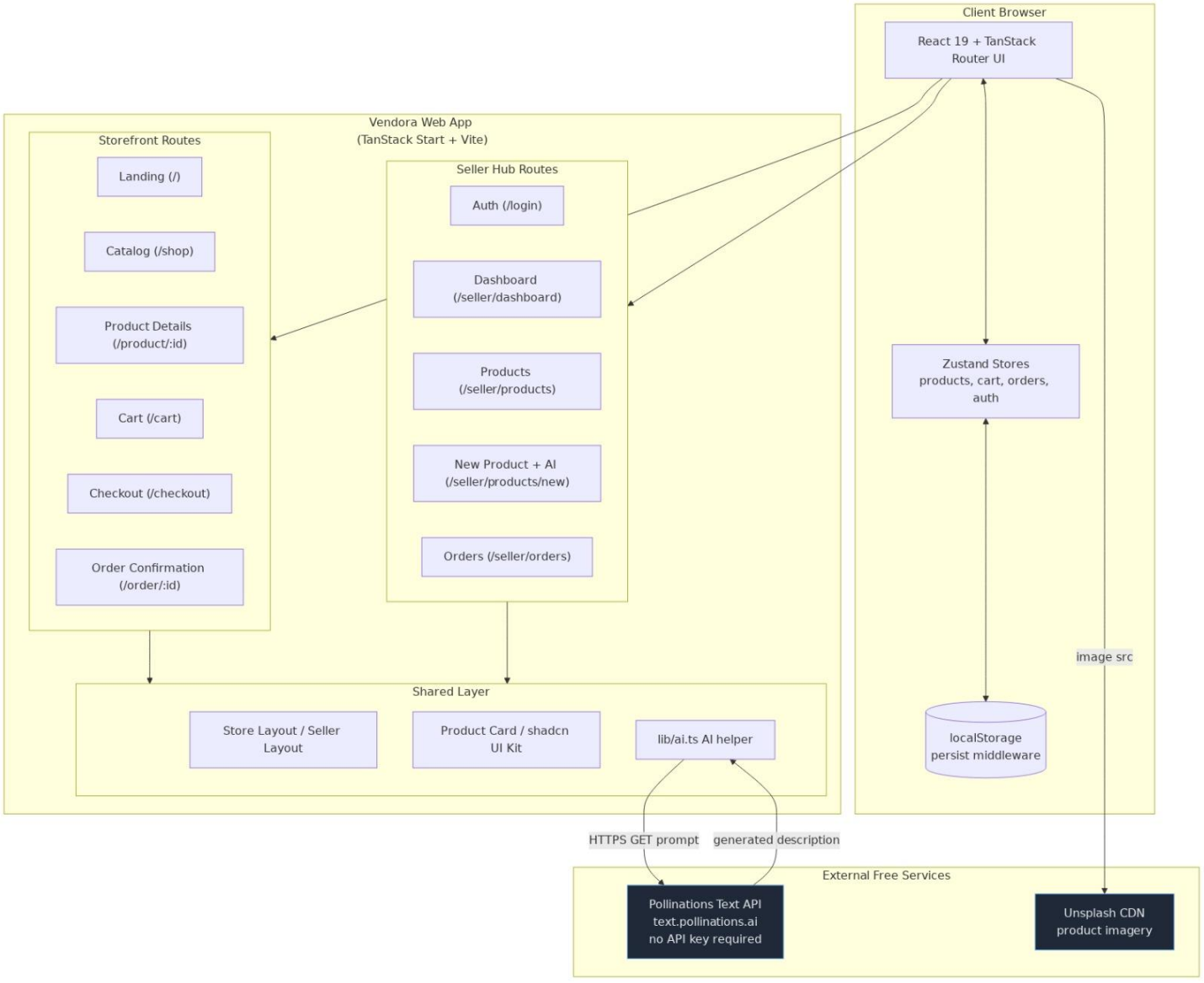
	Zustand	A lightweight state management library used to handle the user's authentication status and persistent shopping cart data across different pages.
Database / Persistence	**Browser localStorage**	Acts as the primary data store for this MVP phase. By using the **Zustand persist middleware**, all data (products, orders, auth) remains saved locally on the user's device, enabling a fully functional demo without external database costs.
External / AI Services	**Pollinations.ai**	A free, open-source AI text generation engine. It allows Vendora to provide the "AI Description Assistant" feature without requiring expensive API keys or complex backend integrations.
	Unsplash CDN	Provides high-quality, static image URLs to ensure the storefront looks professional and aesthetically pleasing during the demonstration.
Development Tools	**Bun**	An all-in-one JavaScript runtime and package manager. It provides significantly faster installation and execution speeds compared to NPM or Yarn, which is critical for meeting the 30-day "project crashing" timeline.
	Vite 7	The next-generation build tool that provides an extremely fast development environment with Hot Module Replacement (HMR), allowing the team to see code changes instantly.
	TypeScript (Strict Mode)	Ensures type safety across the entire codebase. This reduces runtime errors and makes the code "self-documenting," which is essential for a team working under tight deadlines.
Design & Diagram Tools	**Figma**	Used for collaborative wireframing and high-fidelity UI/UX design. It allowed the team to finalize the "minimalist luxury" look before writing a single line of CSS.
	draw.io / Lucidchart	Used to create the WBS, Feature Interaction Flows, and DFDs. Its browser-based collaboration allows for quick exports of diagrams for this documentation.
Testing Tools	**Chrome DevTools**	Essential for debugging React components, inspecting the Tailwind CSS layout, and monitoring the **Network tab** to ensure AI requests to Pollinations.ai are successful.

	Sonner	A toast notification library used to provide immediate visual feedback to users (e.g., "Product Added to Cart" or "Login Successful").
Documentation Tools	**MS Word / Google Docs**	Used to compile the FRS, Business Case, and Project Charter. It allows for structured formatting and easy conversion to the final PDF submission required for the university.

Reason for Architecture Style

Architecture	Reason
Server-Side Rendering (SSR) & Edge Computing	By utilizing **TanStack Start** on **Cloudflare Workers**, the system shifts processing to the "edge" (closer to the user). This eliminates the delay of traditional centralized servers, ensuring the "luxury storefront" loads instantly and remains highly performant for a global audience.
Component-Based Modular Architecture	Built with **React 19**, the system treats every UI element (like the AI-generated product cards or the cart drawer) as an independent module. This allows the team to develop, test, and update individual features without risking "project crashing" across the entire site.
Local-First Data Persistence	Leveraging **Zustand with localStorage** creates a resilient, zero-latency experience. Since data is managed directly in the browser, the app remains fully functional and fast even if external network requests are slow, which is critical for a smooth MVP demonstration.
Decoupled AI Service Integration	The **Pollinations.ai** logic is isolated from the core UI. This decoupling means that the AI text generation can be called asynchronously, preventing the interface from freezing while the description is being drafted, thus maintaining a high-end user experience.
Strict Type-Driven Architecture	Using **TypeScript** as the foundational layer ensures that data flowing between the frontend and backend functions is consistent. This architectural choice was vital during the recovery from the planning phase crash, as it caught integration bugs automatically before they reached production.

Architecture Diagram



Block Diagram